

Fundamentos da Retropropagação, Norma do Erro & Primeira MLP

Aula 02: Como Redes Neurais Aprendem com Seus Erros e Previsão de Churn

Faculdade Infnet

Redes Neurais Profundas (Deep Learning e Visão Computacional)

Data: 2026-07-29

Roteiro & Objetivos da Aula

Quatro blocos fundamentais para dominar como as redes aprendem

01

01

O Conceito de Retropropagação

Entendendo a ideia intuitiva do aprendizado por correção de erros.

02

02

A Norma do Erro (Demonstração)

Simulações visuais da régua de distância e do perigo do cancelamento.

03

03

Ciclo em 5 Passos no PyTorch

Do conceito teórico às 5 linhas de código do loop de treinamento.

04

04

Primeira MLP (SaaS Churn)

Construindo uma rede multicamadas para prever cancelamentos.

O Conceito Intuitivo da Retropropagação (Backpropagation)

Como uma rede neural aprende corrigindo os próprios erros passo a passo

1. Passagem Direta (Forward)

$$\hat{y} = f(X, w)$$

O modelo recebe as entradas e gera um palpite ou estimativa inicial.

2. Medição do Erro (Norma)

$$e = \|y - \hat{y}\|$$

Comparamos a estimativa com a resposta real para saber o tamanho do desvio.

3. Retropropagação (Backward)

Atribuição de Culpa

Caminhamos do fim para o início identificando a contribuição de cada peso no erro.

4. Ajuste dos Pesos

$$w_{\text{novo}} = w - \text{ajuste}$$

Modificamos os pesos para que o próximo palpite seja mais preciso.

Visualizando a Norma do Erro: A Régua de Distância

Mova a estimativa ($\hat{y}$) para ver em tempo real como a norma mede a gravidade da diferença

Demonstrador Visual da Norma do Erro: A Régua de Distância

Mova os controles para ver como a Norma mede a gravidade da diferença entre a estimativa e o real.

Norma L1 (Absoluta)

Norma L2 (Euclidiana)

Erro Quadrático (MSE)

 Valor Real Desejado (y): **80% (0.80)**



 Palpite do Modelo ($\hat{y}$): **20% (0.20)**



CÁLCULO DA NORMA EM TEMPO REAL:

$$\|y - \hat{y}\|_2 = \sqrt{(0.60)^2} = 0.60$$

0% (Sem Churn)

RÉGUA DE DISTÂNCIA

100% (Churn)



Palpite $\hat{y}$: 20%

Distância: 60%

Meta y : 80%

VELOCÍMETRO DA NORMA DO ERRO:

0.600



Alerta: Erro alto! A norma aciona uma forte correção nos pesos

Por Que Usamos a Norma? A Armadilha do Cancelamento

Compare a soma simples sem norma vs. o diagnóstico real com a norma do erro

⚠ A Armadilha do Cancelamento: Por Que a Norma é Obrigatória?

Compare o cálculo do erro sem a norma vs. com a norma em duas amostras com desvios opostos.

✗ Soma Sem Norma (Perigoso)

✓ Com Norma do Erro (Correto)

Cliente A (Falta de Acesso)

Palpite do Modelo ($\hat{y}_1$): 0.40 (40%)

Cancelou ($y_1 = 1.0$)

Erro Simples: ($\hat{y}_1 - y_1$) = 0.40 - 1.00 = -0.60

Cliente B (Cliente Ativo)

Palpite do Modelo ($\hat{y}_2$): 0.60 (60%)

Ativo ($y_2 = 0.0$)

Erro Simples: ($\hat{y}_2 - y_2$) = 0.60 - 0.00 = +0.60

CÁLCULO COM A NORMA DA DISTÂNCIA (ABSOLUTA/QUADRÁTICA):

$$\text{ErroTotal} = | - 0.60 | + | + 0.60 | = 0.60 + 0.60 = 1.20$$

✓ DIAGNÓSTICO CORRETO: Erro Total = 1.20! A rede percebe o defeito e dispara o ajuste dos pesos.

A Mecânica da Retropropagação em 3 Etapas Calmas

Entendendo a transmissão do sinal de erro da saída até as entradas

Etapa 1: Erro na Camada de Saída

$$E_{\text{saída}} = \text{Norma}(y, \hat{y})$$

Calculamos a diferença final entre a estimativa e o objetivo real.

Etapa 2: Distribuição de Responsabilidade

Sinal de Erro $\rightarrow$ Camadas Ocultas

O erro é repartido entre os neurônios que geraram a saída proporcionalmente a seus pesos.

Etapa 3: Atualização Local dos Pesos

$$\Delta w \propto \text{Entrada} \times \text{Erro do Neurônio}$$

Cada peso individual é ajustado de acordo com seu papel no erro final.

A Ferramenta Matemática: A Regra da Cadeia

Como multiplicamos pequenas reações locais para encontrar o ajuste final

$$\frac{\partial \text{Erro}}{\partial w} = \underbrace{\frac{\partial \text{Erro}}{\partial \hat{y}}}_{\text{Norma do Erro}} \cdot \underbrace{\frac{\partial \hat{y}}{\partial z}}_{\text{Efeito da Ativação}} \cdot \underbrace{\frac{\partial z}{\partial w}}_{\text{Sinal de Entrada (x)}}$$

$$\frac{\partial \text{Erro}}{\partial \hat{y}}$$

Norma da distância: quanto o erro geral muda se a estimativa final mudar.

$$\frac{\partial \hat{y}}{\partial z}$$

Ativação: quanto a estimativa muda quando variamos o resultado bruto do neurônio.

$$\frac{\partial z}{\partial w}$$

Entrada x: a contribuição direta do peso w na combinação linear $z = w \cdot x + b$.

Simulador Interativo: Passagem Direta vs. Retropropagação

Altere x, w e b para ver a geração do valor z, a norma y e o cálculo dos gradientes

⚡ Grafo de Computação (DAG): Da Entrada à Saída ($\hat{y}$) e à Função de Perda (L)

Ajuste a entrada (x), peso (w) e meta real (y) para ver o cálculo dos gradientes dos parâmetros treináveis.

Passagem Direta (Forward →)

Retropropagação (Backward ←)



Entrada (x)

x = 2.0

Dado Fixo (Não Treinável)

Peso w (requires_grad)

w = 3.0

Parâmetro Treinável

Viés b (requires_grad)

b = 1.0

Parâmetro Treinável



Forward (→)

Soma Ponderada (z)

$z = w \cdot x + b$

7.0



Forward (→)

Saída da Rede ($\hat{y}$)

Palpite Estimado

$\hat{y} = 7.0$



Forward (→)

Nó de Perda / Loss (L)

$L = (\hat{y} - y)^2$

L = 49.0

🔍 Gradientes dos Parâmetros Treináveis (w e b):

O Ciclo Sagrado de Treinamento (5 Passos)

A sequência exata que você escreverá em 100% dos projetos em PyTorch

1. `otimizador.zero_grad()`

Zera gradientes acumulados

Limpa os gradientes calculados na época anterior.

2. `pred = modelo(X)`

Passagem Direta (Forward)

Passa os dados pela rede para gerar a estimativa $\hat{y}$.

3. `loss = criterio(pred, Y)`

Norma do Erro (Calcula Loss)

Mede a norma da distância entre a estimativa $\hat{y}$ e a meta Y .

4. `loss.backward()`

Retropropagação (Backward)

Executa a Regra da Cadeia trazendo o erro da saída para os pesos.

5. `otimizador.step()`

Atualização dos Pesos

Ajusta os pesos: $w = w - \text{learning_rate} * \text{gradiente}$.

Simulador Interativo do Loop de Treinamento

Clique em "Avançar Passo" e acompanhe a transição dos tensores nas 5 etapas

O Ciclo Sagrado de Treinamento em 5 Passos do PyTorch

Clique no botão de avanço para executar passo a passo uma iteração do loop de treinamento.

 Reiniciar

Avançar Passo →

PASSO 1

`otimizador.zero_grad()`

PASSO 2

`pred = modelo(X)`

PASSO 3

`perda = criterio(pred, Y)`

PASSO 4

`perda.backward()`

PASSO 5

`otimizador.step()`

PASSO 1 DE 5

1. Zerar Gradientes Acumulados

Por padrão, o PyTorch acumula gradientes em `.grad` em cada chamada `backward`. Zeramos os gradientes anteriores antes de iniciar uma nova época para evitar acúmulo indesejado.

```
# Código PyTorch em execução:  
otimizador.zero_grad()
```

Estado do Grafo & Tensores

Parâmetro (`w`): `w = 2.50`

Gradiente (`w.grad`): `grad = 0.00 (Zered!)`

Perda (`Loss`): `Loss anterior limpa`

Função de Perda Binária: Binary Cross-Entropy

A norma probabilística para medir erros em problemas de classificação (0 ou 1)

$$L(y, \hat{y}) = - [y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

y

Rótulo real da amostra (0 para cliente ativo, 1 para Churn/cancelado).

$\hat{y}$

Probabilidade prevista pela rede neural (estimativa da saída, entre 0 e 1).

`nn.BCELoss()`

Implementação em PyTorch da norma de perda por Entropia Cruzada Binária.

O Otimizador: Gradiente Descendente Estocástico

Como o `optim.SGD` usa o gradiente da retropropagação para ajustar os pesos

1. Direção Oposta ao Gradiente

$$-\nabla L(w)$$

O gradiente indica para onde o erro cresce; por isso andamos na direção contrária.

2. Escalonamento pela Taxa de Aprendizado (η)

$$\eta \cdot \text{Gradiente}$$

A taxa de aprendizado (learning rate) controla o tamanho do passo de ajuste.

3. Atualização do Peso

$$w_{\text{novo}} = w_{\text{atual}} - \eta \cdot \text{Gradiente}$$

O peso é atualizado para reduzir a norma do erro na próxima época.

Laboratório de Learning Rate: Hiperparâmetro Crítico

O que acontece quando a taxa de aprendizado (η) é grande ou pequena demais?

TAXA MUITO ALTA (ex: $lr = 10.0$)

Passos Gigantes & Divergência

- ▶ O passo de ajuste pula o ponto de erro mínimo.
- ▶ A norma da perda (Loss) começa a CRESCER em vez de diminuir.
- ▶ O modelo desestabiliza e pode gerar erros numéricos (NaN).
- ▶ Impossibilita o aprendizado da rede.

TAXA MUITO BAIXA (ex: $lr = 0.0001$)

Passos Microscópicos & Lentidão

- ▶ Os ajustes nos pesos são tão pequenos que o erro quase não cai.
- ▶ O modelo exige milhares de épocas e horas de treinamento.
- ▶ Pode ficar preso em valas rasas de erro (mínimos locais).
- ▶ Desperdício de tempo de computação.

Laboratório Interativo: Curvas de Aprendizado

Selecione a Taxa de Aprendizado (η) e analise a queda da norma da Loss



Laboratório de Hiperparâmetros: O Impacto do Learning Rate (η)

Selecione uma Taxa de Aprendizado para observar o comportamento da curva de erro (Loss) durante a otimização.

$\eta = 10.0$

Explosão / Oscilação

$\eta = 0.1$

Ideal / Rápido

$\eta = 0.01$

Suave / Seguro

$\eta = 0.0001$

Lentidão Extrema



Curva de Perda (Loss) vs Épocas de Treinamento



Diagnóstico do Aprendizado:

Taxa ótima! O modelo converge de forma rápida e estável para a perda mínima.



Dica prática: Comece otimizadores como Adam com $\eta = 0.001$ ou SGD com $\eta = 0.01 / 0.1$ e monitore a curva de perda!

Caso Prático de Negócios: Previsão de Churn em SaaS

Identificando antecipadamente clientes com alto risco de cancelamento



- ▶ **O Desafio Corporativo:** Cancelamentos (Churn) reduzem a receita recorrente da empresa de software.
- ▶ **Variável Alvo (Y):** `Churn` (1 = Cancelou a assinatura, 0 = Cliente Continua Ativo).
- ▶ **Variáveis de Entrada (X):** Tempo de Contrato (meses), Faturamento (\$), Chamados ao Suporte e Dias Inativo.
- ▶ **Objetivo:** Treinar uma MLP para estimar a probabilidade de Churn e alertar o time comercial.

Arquitetura da Primeira MLP (Multi-Layer Perceptron)

Encadeando camadas ocultas para aprender padrões complexos de Churn

Camada de Entrada (Input)

$$X \in \mathbb{R}^4$$

4 atributos do cliente (Contrato, Faturamento, Suporte, Dias Inativo).

Camada Oculta (Hidden)

$$h = \text{ReLU}(W_1 X + b_1) \in \mathbb{R}^8$$

8 neurônios ocultos com ativação ReLU para capturar combinações não-lineares.

Camada de Saída (Output)

$$\hat{y} = \text{Sigmoid}(W_2 h + b_2) \in [0, 1]$$

1 neurônio com Sigmoid gerando a estimativa de probabilidade de Churn.

Simulador Interativo da MLP de Churn SaaS

Mova os sliders do perfil do cliente e veja a ativação dos neurônios e o risco estimado

Simulador da MLP de Churn de Clientes SaaS (Telco)

Altere as métricas do cliente abaixo e veja a ativação da camada oculta (ReLU) e a predição da camada de saída (Sigmoid).



ENTRADA (4 FEATURES)

Contrato: 6m

Faturam.: R\$ 150

Suporte: 7 cham.

Inativo: 18d

CAMADA OCULTA (8 NEURÔNIOS RELU)

h1
2.16
ATIVADO

h2
2.22
ATIVADO

h3
0.00
INATIVO

h4
0.00
INATIVO

h5
1.48
ATIVADO

h6
0.90
ATIVADO

h7
1.64
ATIVADO

h8
2.10
ATIVADO

SAÍDA (SIGMOID)

PROBABILIDADE DE CHURN

93%

ALTO RISCO DE CHURN!

Implementação da Classe MLPChurn em PyTorch

Código Python claro e estruturado herdeiro de `nn.Module`

```
class MLPChurn(nn.Module) :
```

self.camada_culta

`nn.Linear(in_features=4, out_features=8)`

self.relu

`nn.ReLU()` - função de ativação não-linear da camada oculta

self.camada_saida

`nn.Linear(in_features=8, out_features=1)`

self.sigmoid

`nn.Sigmoid()` - comprime a saída em uma probabilidade entre 0 e 1

Resultados do Treinamento no Dataset Telco Churn

Acompanhando a queda da norma do erro e a evolução da acurácia

01

01

Padronização dos Dados

Transformação Z-score nas entradas para equilibrar as escalas dos atributos.

02

02

Otimizador Adam

Uso do optim.Adam(lr=0.01) com adaptação automática de taxa de aprendizado.

03

03

Queda da Norma do Erro

A Loss (BCELoss) caiu de 0.69 para menos de 0.25 ao longo das épocas.

04

04

Acurácia Alcançada

O modelo atingiu mais de 88% de acertos na classificação dos clientes.

Teste de Consolidação & Próximos Passos

Valide seu aprendizado sobre retropropagação, norma do erro e MLPs

Teste Seus Conhecimentos: Autograd & Treinamento de Redes Neurais

Responda às questões abaixo para consolidar os conceitos aprendidos nesta aula.

Questão 1: Por que executamos `otimizador.zero_grad()` no início de cada época no PyTorch?

- A) Para zerar os pesos e reiniciar a rede neural do zero.
- B) Porque por padrão o PyTorch acumula gradientes em `.grad` a cada chamada `backward`.
- C) Para apagar os dados de entrada da memória GPU.
- D) Para reiniciar a taxa de aprendizado (learning rate).

Questão 2: Qual é o papel do motor Autograd durante a execução de `loss.backward()`?

- A) Ajustar os pesos diretamente somando a taxa de aprendizado.
- B) Calcular a acurácia do modelo no conjunto de teste.
- C) Percorrer o Grafo Direcionado Acíclico (DAG) no sentido inverso calculando derivadas via Regra da Cadeia.